Übungsblatt 5: Inter-Task-Kommunikation und Interrupts

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Musterlösung

Zielsetzung

Zwei Versuche auf dem Cortex-M4 (FreeRTOS, CMSIS-RTOS2). Teil A transportiert Nachrichten über eine Queue und zeigt einen definierten Überlauf. Teil B weckt einen Worker aus der EXTI-ISR. Board **NUCLEO-H755ZI-Q** nur, wenn das Cube-Projekt es ist. UM2408: User-Button standardmäßig PC13, alternativ PA0; gelbe LD2 an PE1, Brücken prüfen. Im Kursrepository liegt kein Firmwareprojekt. Tick, NVIC-Grouping und EXTI-Zielkern dort ablesen. Ein Kernel auf dem CM4 ist kein gemeinsamer Scheduler mit dem CM7.

Aufgabe 1: Queue mit Sequenz

1. FreeRTOS nur für den CM4, CMSIS_V2. HAL-Zeitbasis wie in VL3 wählen und begründen, nicht als Naturgesetz.
2. `osMessageQueueNew` für 8 Elemente `uint32_t` oder dieses Struct:

```
typedef struct {
    uint32_t sample;
    uint32_t seq;
} SensorMsg;
```

3. Producer: synthetische Folge `sample = seq`, Put mit `msg_prio = 0`. Das ist im FreeRTOS-Adapter FIFO; der Adapter ignoriert die Nachrichtenpriorität. Bei `osOK` ist die Kopie eingereiht, sonst Drop-Zähler. Get nur bei `osOK` auswerten.
4. Periode und Queuegröße in Text und Code gleich halten. `osDelay` ist Tickwarte, kein exaktes 1-ms-Raster. Für ein Raster `osDelayUntil` wie in VL3, Überlast nicht still aufholen.
5. Überlastvariante: Producer schneller als Consumer, Try-Put. Erwartung: Lücken in `seq`, nicht ein stilles Weiterzählen beim Consumer.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 1

```
void Producer(void *argument)
{
    SensorMsg m = {0, 0};
    (void)argument;
    for (;;) {
        m.seq++;
        m.sample = m.seq;
        if (osMessageQueuePut(q, &m, 0, 0) != osOK)
            drops++;
        osDelay(10);
    }
}

void Consumer(void *argument)
{
    SensorMsg m;
    (void)argument;
    for (;;) {
        if (osMessageQueueGet(q, &m, NULL, 50) != osOK)
            continue;
        last_seq = m.seq;
        osDelay(50);
    }
}
```

`osMessageQueueGetCount` nur ansehen, nicht als Reservierung vor Put/Get. Lehrbilanz der Vorlesung (N=10, Put jede ms, Get alle 5 ms) ist ein anderes Modell: erster Fehl-Put bei 13 ms, 13 Erfolge, 7 Verwürfe, 4 Gets, 9 Rest. Nicht als Messung dieses 10/50-ms-Codes eintragen.

Aufgabe 2: Latest-Value ist kein Try-Put

1. Queue der Länge 1. Put 10 mit Blockieren, bis es `osOK` ist.
2. Put 20 mit Timeout 0. Erwartung: Fehler, gespeichert bleibt 10.
3. Eine echte Latest-Value-Variante getrennt bauen: native Queue nur in einem kleinen Modul mit `xQueueOverwrite` **oder** ein geschützter Snapshot mit Versionszähler im Task. `osMessageQueueId_t` nicht nach `QueueHandle_t` casten. Kein Get-dann-Put als atomares Overwrite.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 2

Try-Put auf der vollen 1er-Queue lässt 10 stehen. Overwrite ersetzt 10 durch 20. Mehrere Leser per Peek oder Snapshot sehen den Stand; Get würde ihn nur einem Consumer geben.

Aufgabe 3: EXTI-Priorität und leerer Semaphor

1. Button als EXTI auf dem Kern, dessen Vektor das RTOS sieht. Bei CM4-Kernel nicht den CM7-Callback unkoordiniert dieselbe Leitung bedienen lassen.
2. NVIC: Haken **und** logische Priorität. Kleinere Zahl = höhere IRQ-Dringlichkeit. Die Zahl muss auf der erlaubten Seite von `configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY` liegen. Den geschobenen Registerwert nicht mit der Cube-Eingabe verwechseln. `__NVIC_PRIO_BITS` und Grouping aus dem Projekt. Nicht „immer Priorität 5“.
3. `ButtonSem = osSemaphoreNew(1, 0, NULL)` nach `osKernelInitialize`, Handle prüfen. Threads erzeugen, dann `osKernelStart`. EXTI erst aktivieren, wenn der Kernel läuft, zum Beispiel im ersten Thread. Danach hängende Pending-Flags nur nach diesem Vertrag behandeln.
4. Pfad nicht verdoppeln: `EXTI15_10_IRQHandler` ruft `HAL_GPIO_EXTI_IRQHandler`. Der Callback prüft die Pin-Maske und released. HAL quittiert. Kein zweites Flag-Löschen, kein `printf`, kein Mutex.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 3

```
void HAL_GPIO_EXTI_Callback(uint16_t pin)
{
    if (pin != GPIO_PIN_13) return;
    if (osSemaphoreRelease(ButtonSem) != osOK)
        isr_fail++;
}
```

Pin 13 nur, wenn PC13 der aktive User-Button ist. `isr_fail` nicht aus Thread und ISR gleichzeitig mit Read-Modify-Write ohne Vertrag erhöhen; für die Übung einen `volatile`-Zähler nur als Diagnose markieren und die Grenze nennen.

Worker:

```
void ButtonTask(void *argument)
{
    (void)argument;
    for (;;) {
        if (osSemaphoreAcquire(ButtonSem, osWaitForever) != osOK)
```

```
            continue;
        HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_1);
    }
}
```

PE1 nur nach Prüfung von LD2. UART-Text nur über den Mutex-Pfad aus VL4.

Vergleich, ausdrücklich kennzeichnen: `HAL_Delay(100)` im High-Worker ist Busy-Wait. Niedrigere Threads stocken für diese Last. Eine höhere Task kann trotzdem präemptieren. Die LED beweist keine Deadline.

Drei Releases vor einem Acquire bei max count 1: höchstens ein Token, weitere Releases können scheitern. Das ist Zusammenfassen, kein lückenloser Zähler. Prellen extra behandeln: Pegel stabil lesen, Press und Release merken. 100 ms Wartezeit löscht keine schon gespeicherten Tokens.

Option: Event-Queue, Put aus der ISR mit Timeout 0, Drop-Zähler. Auch die ist endlich.

Aufgabe 4: Auswertung

Punkt	Beobachtung
Semaphor-Startcount 0	
NVIC-Priorität und Grouping	
Tickrate	
Workerpriorität	
Drop- und ISR-Fehlerzähler	
ISR-Dauer, Startlatenz, Completion	
Heartbeat unter der Busy-Wait-Variante	
Zahl verarbeiteter Ereignisse	

Die 16 µs der Vorlesung sind synthetisch (Flanke 0, Eintritt 3, Ready 8, ISR-Ende 12, Lauf 16). Nicht als Boardwert eintragen. Messen mit GPIO oder Trace; `printf` danach. Ohne Logic Analyzer die Zeiten leer lassen.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 4

Soll ohne Messung: leerer Semaphor weckt erst nach Release; zu dringliche EXTI-Priorität darf die API nicht rufen; voller binärer Semaphor verwirft weitere Flanken; Try-Put auf voller Queue verwirft die neue Nachricht und lässt die alte liegen. Ein erfolgreicher Build ist keine gehaltene Frist.